

Home Assignment 3

Friday, 7 November 2025 15:29

Solution Presented by: Akanksha Vijayvergiya

Concurrency vs Parallelism

Concurrency

- Concurrency means that multiple tasks make progress during the same period of time by interleaving their execution.
- The focus is on managing multiple tasks efficiently to maintain responsiveness.
- It can work even on a single-core CPU, where the processor rapidly switches between tasks using context switching.
- Main Goal is Responsiveness — ensuring the system remains active and can handle multiple activities smoothly.

Example Use Cases:

- Database systems: serving multiple queries concurrently.
- Web servers or Application servers: handling multiple requests.

Java APIs for Concurrency:

- Thread, Runnable, etc.

Illustration:

- In a single-core CPU, tasks are interleaved, not truly simultaneous.

(one core, interleaved tasks)

Parallelism

- Parallelism means that multiple tasks run literally at the same time on different CPU cores or processors.
- The focus is on speeding up computation and improving performance by dividing work among multiple cores.
- Primary Goal is Performance — completing tasks faster through simultaneous execution.

Example Use Cases:

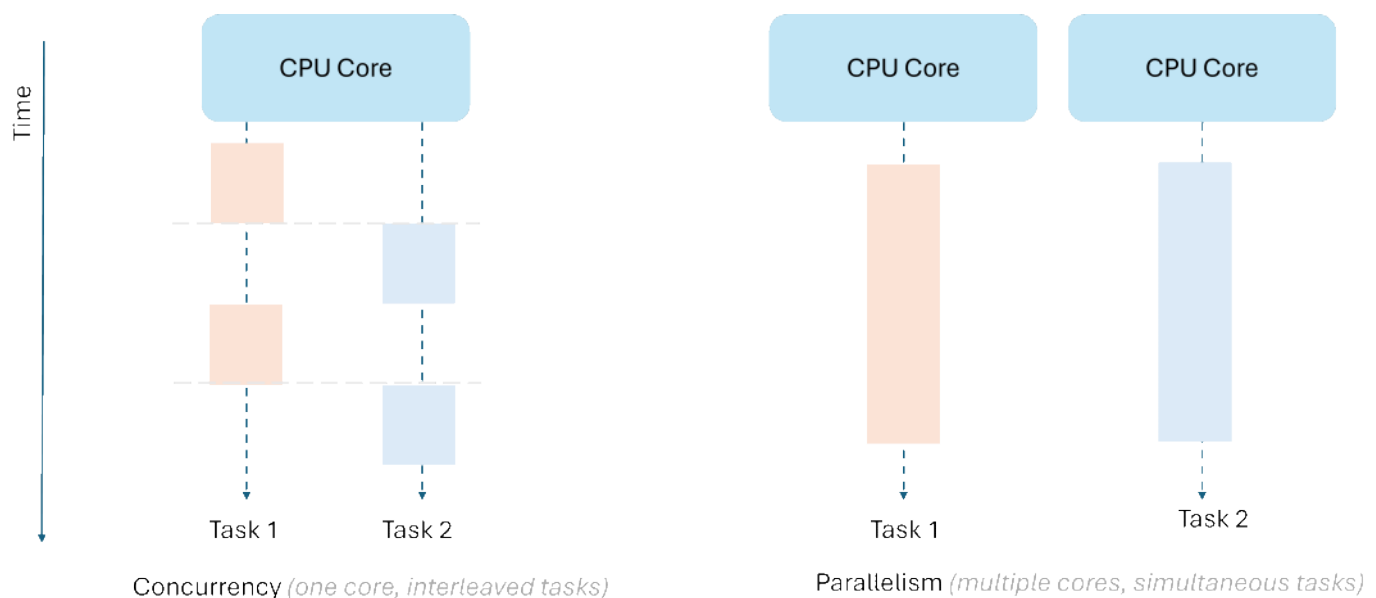
- Machine learning and data processing.

Illustration:

- In multi-core CPUs, tasks run simultaneously on different cores.

(multiple cores, simultaneous tasks)

Concurrency is about dealing with many things at once, while Parallelism is about doing many things at once.



Java Concurrency Models

Model	Description	Examples	Pros	Cons
Shared State	Threads share data and coordinate with locks (synchronized, wait/notify)	Thread, Runnable, synchronized	• Simple to start threads • Direct data access	• Race conditions, deadlocks • Hard to debug & scale
Thread Pool / Executors	Manages a pool of threads to run tasks efficiently	ExecutorService, Future	• Automatic thread reuse • Better performance & scalability	• Still uses shared state • Pool size tuning needed
Separate State / Async	Each task has its own data; uses async or message passing	CompletableFuture, Akka, Vert.x	• Non-blocking & scalable • Fewer concurrency bugs	• Complex error handling • Harder debugging
Virtual Threads (Project Loom)	Lightweight JVM-managed threads that scale massively	Thread.ofVirtual(), Loom API	• Millions of threads • Simple blocking-style code	• New feature (less mature) • Some tools not optimized yet